> 这道题需要学习的东西太多了……**我写了三个笔记，分别是MySQL、docker、nginx，在一个压缩包里**

PART 1

TASK 1

> ### 前言
>
> 既然要创建一个书籍信息数据库，那么就先来列一个小提纲吧
>
> 需要多少字段？需要哪些字段？
>
> 书名肯定要，图书管理系统那还有书籍编号才好管理，还有一些书相关的信息和借阅状态
>
> 所以，我把数据库定为6个字段
>
> 分别为**编号、书名、作者、ISBN、出版社、借阅状态**
>
> 来吧

- **创建数据库**

用create database创建一个名为library的数据库

```
mysql> create database library DEFAULT CHARSET utf8mb4;
Query OK, 1 row affected (0.02 sec)

mysql> show databases
    -> ;
+--------------------+
| Database           |
+--------------------+
| information_schema |
| library            |
| mysql              |
| performance_schema |
| sakila             |
| sys                |
| world              |
+--------------------+
7 rows in set (0.00 sec)
```

切换到这个数据库之后，就可以开始建表了

- **创建信息表**

按照前言写的字段设计思路，先把代码写好：

> 有以下几点细节：
>
> 1. id是唯一的，需要作为主键
> 2. name,author,isbn这些为空不太好
> 3. 借阅状态那里我也不知道怎么表示才好(

```
create table books(
    id int PRIMARY KEY COMMENT '编号',
    name varchar(100) NOT NULL COMMENT '书名',
    author varchar(50) NOT NULL COMMENT '作者',
    isbn varchar(20) NOT NULL COMMENT 'ISBN号',
    press varchar(50) COMMENT '出版社',
    borrow tinyint COMMENT '借阅状态，0未借阅，1已借阅'
) COMMENT '图书管理系统';
```

输入之后来看看，也是创建成功了(怎么老是忘记这个分号)

```
mysql> desc books
    -> ;
+--------+--------------+------+-----+---------+-------+
| Field  | Type         | Null | Key | Default | Extra |
+--------+--------------+------+-----+---------+-------+
| id     | int          | NO   | PRI | NULL    |       |
| name   | varchar(100) | NO   |     | NULL    |       |
| author | varchar(50)  | NO   |     | NULL    |       |
| isbn   | varchar(20)  | NO   |     | NULL    |       |
| press  | varchar(50)  | YES  |     | NULL    |       |
| borrow | tinyint      | YES  |     | NULL    |       |
+--------+--------------+------+-----+---------+-------+
```

- **插入数据**

  接下来插入三本书的数据，用insert into

  ```
  insert into books(id,name,author,isbn,press,borrow) values (1,'《傲慢与偏见》','Jane Austen','9787544798396','译林出版社',0),(2,'《希腊棺材之谜》','Ellery Queen','9787802254930','新星出版社',1),(3,'《活着》','余华','9787506365437','作家出版社',0);
  ```

TASK 2

刚刚插入了三本书的数据，这里来进行对数据的查询修改删除

- ## 查询所有书籍信息

  查询用SELECT，因为是所有信息，字段用通配符*代替

```
mysql> SELECT * from books;
+----+-----------------+-------------+---------------+------------+--------+
| id | name            | author      | isbn          | press      | borrow |
+----+-----------------+-------------+---------------+------------+--------+
|  1 | 《傲慢与偏见》  | Jane Austen | 9787544798396 | 译林出版社 |      0 |
|  2 | 《希腊棺材之谜》| Ellery Queen| 9787802254930 | 新星出版社 |      1 |
|  3 | 《活着》        | 余华        | 9787506365437 | 作家出版社 |      0 |
+----+-----------------+-------------+---------------+------------+--------+
```

完成了！

- ## 查询指定书籍的信息

  这里我想查询《希腊棺材之谜》这本书的信息

  加一个WHERE就好（怎么又忘记分号了）

```
mysql> SELECT * FROM books WHERE name='《希腊棺材之谜》'
    -> ;
+----+-----------------+-------------+---------------+------------+--------+
| id | name            | author      | isbn          | press      | borrow |
+----+-----------------+-------------+---------------+------------+--------+
|  2 | 《希腊棺材之谜》| Ellery Queen| 9787802254930 | 新星出版社 |      1 |
+----+-----------------+-------------+---------------+------------+--------+
1 row in set (0.01 sec)
```

- ## 修改某个书籍的信息

  修改使用UPDATE，这里我修改《活着》的出版社，改为新星出版社

```
mysql> SELECT * from books;
+----+-----------------+-------------+---------------+------------+--------+
| id | name            | author      | isbn          | press      | borrow |
+----+-----------------+-------------+---------------+------------+--------+
|  1 | 《傲慢与偏见》  | Jane Austen | 9787544798396 | 译林出版社 |      0 |
|  2 | 《希腊棺材之谜》| Ellery Queen| 9787802254930 | 新星出版社 |      1 |
|  3 | 《活着》        | 余华        | 9787506365437 | 作家出版社 |      0 |
+----+-----------------+-------------+---------------+------------+--------+
3 rows in set (0.00 sec)

mysql> UPDATE books SET press='新星出版社' WHERE name='《活着》';
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> SELECT * from books;
+----+-----------------+-------------+---------------+------------+--------+
| id | name            | author      | isbn          | press      | borrow |
+----+-----------------+-------------+---------------+------------+--------+
|  1 | 《傲慢与偏见》  | Jane Austen | 9787544798396 | 译林出版社 |      0 |
|  2 | 《希腊棺材之谜》| Ellery Queen| 9787802254930 | 新星出版社 |      1 |
|  3 | 《活着》        | 余华        | 9787506365437 | 新星出版社 |      0 |
+----+-----------------+-------------+---------------+------------+--------+
3 rows in set (0.00 sec)
```

- **删除指定书籍的数据**

  删除使用delete，我这里把《傲慢与偏见》删掉

```
mysql> delete from books where name='《傲慢与偏见》';
Query OK, 1 row affected (0.01 sec)

mysql> SELECT * from books;
+----+------------------+--------------+---------------+------------+--------+
| id | name             | author       | isbn          | press      | borrow |
+----+------------------+--------------+---------------+------------+--------+
|  2 | 《希腊棺材之谜》 | Ellery Queen | 9787802254930 | 新星出版社 |      1 |
|  3 | 《活着》         | 余华         | 9787506365437 | 新星出版社 |      0 |
+----+------------------+--------------+---------------+------------+--------+
2 rows in set (0.00 sec)
```

TASK 3

- **为什么选择一个字段作为唯一标识?**

  我这里选择了图书编号**id作为唯一标识**

  看所有字段，像名字、作者、isbn这些都是可以重复的数据，如果有同名的甚至是同一本书，我们录入图书的记录就没法保持唯一性了

  所以，要设置一个唯一的字段id作为**主键**（PRIMARY KEY），让数据库能够区分他们

- **如果存在两本书籍名称相同的情况，数据库应该如何区分这两条记录?**

  由于书名会重复无法作为主键，我们需要增加一个**主键 id**（虽然我那里的id好像没设置为自增的），作为这条记录在当前表内的唯一标识

  这样一来，不论其他字段内有多少重复数据，总会有一个独特的主键id，让数据库能够区分它们

MySQL的本身特点

除了MySQL那些笔记里常见的关系式数据库、数据存在表里等等特点以外，它还有什么特点呢?

mysql的优点:

1. 结构清晰，管理那些有明确键值关系的数据很有优势
2. mysql支持sql语言，查询、修改等等操作很方便

mysql的缺点?

1. mysql的数据存在表里，看起来清晰，但要是数据量巨大的时候设计不当会出问题，增加维护的成本
2. mysql在数据量过多、查询过于复杂时性能有限
3. 以二维表为核心，适用范围说实话不是很宽泛

所以，由于mysql数据存于表中的特点，它更适合在关系明确的数据存储里面（比如学生信息、图书管理、订单等等)

PART 2

docker版本

```
ellery@ellery-VMware-Virtual-Platform:~$ docker --version
Docker version 29.9.0, build f415da8
```

hellodocker！（提前拉过了）

```
ellery@ellery-VMware-Virtual-Platform:~$ docker run hello-world

Hello from Docker!
```

查看和停止

```
ellery@ellery-VMware-Virtual-Platform:~$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED          STATUS          PORTS
                NAMES
b2d9f6d6e2d1   nginx:alpine   "/docker-entrypoint.…"   35 seconds ago   Up 34 seconds   0.0.0.0:8080->80/tcp, [::
]:8080->80/tcp   elastic_faraday
ellery@ellery-VMware-Virtual-Platform:~$ docker stop b2d9f6d6e2d1
b2d9f6d6e2d1
ellery@ellery-VMware-Virtual-Platform:~$ docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
```

docker容器和镜像到底有什么区别?

镜像和容器本质都是文件，但是他们在**功能、状态、文件特点、数量、数据**等等方面都有不同

在功能上，他们和类与对象的关系十分相似

镜像是一个用来启动的模板，而容器是运行的实例，之前题解用到的话“类是对象的抽象，对象是类的实现”到这里依旧能解释镜像和容器的关系；

在状态和文件特点上，镜像由于是模板，所以它是**只读的**，只有容器才能进行修改

要修改原镜像的内容只能产生新镜像，可以使用容器启动后的可写层进行改动，再把容器转为新的镜像；

在数量关系上，一个镜像可以产生多个容器，和类与对象类似

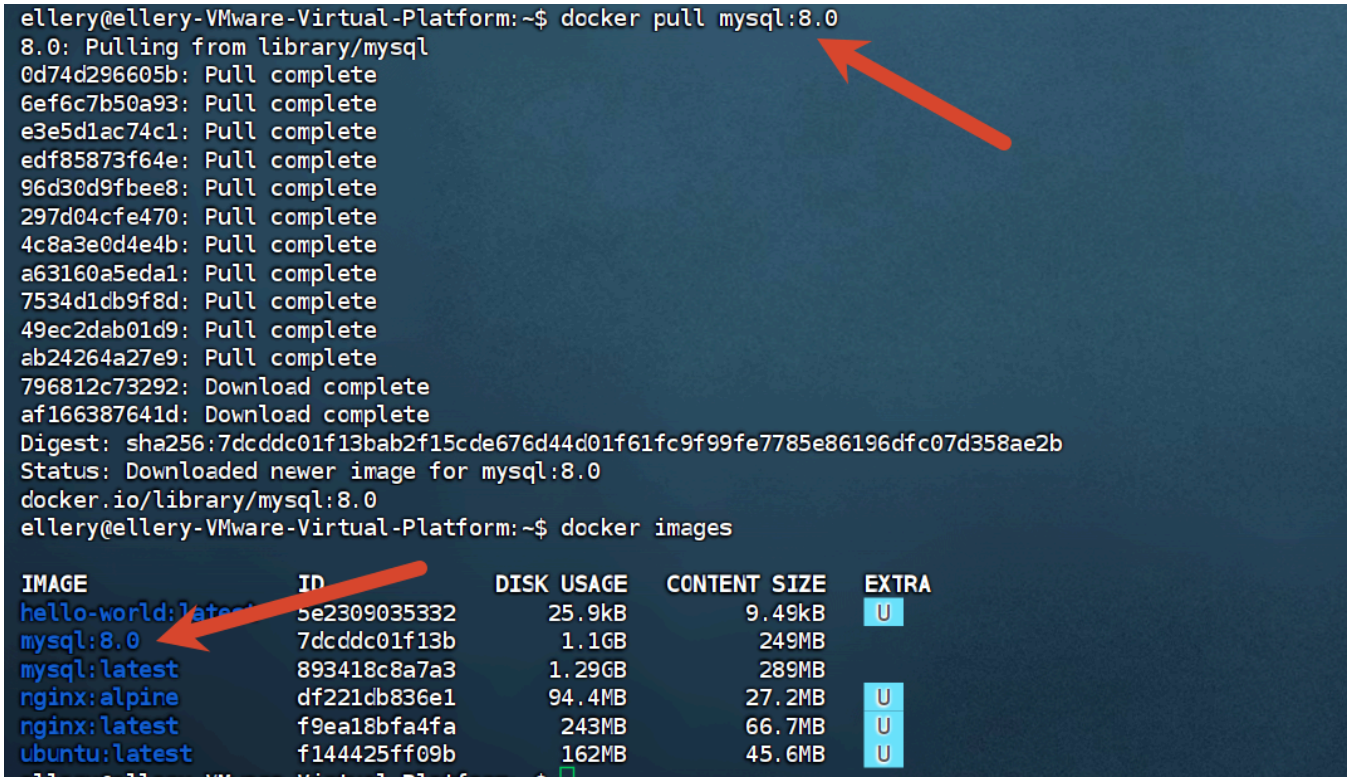
在数据上，镜像会保存数据、文件，而容器一旦删除里面的数据也会消失，除非挂载到数据卷上了存在宿主机里面

用Docker标准化部署快速搭建mysql

获取与部署&数据持久化

> 只有一个mysql可以直接docker run，但是奈何我实在学不会compose（，还是练习一下吧

1. **拉取mysql8.0**（我初学的时候拉了个默认的最新版本，应该比8.0大，所以再拉一次了）



2. **写compose.yaml**

> 这个yaml文件需要写镜像名、容器名字、环境变量（密码和默认数据库）、端口、挂载数据卷
>
> 其中，镜像名用image，容器名container_name，环境变量environment，密码MYSQL_ROOT_PASSWORD，数据库MYSQL_DATABASE，端口ports、挂载volumes

```yaml
services:
 mysql:
  image: mysql:8.0
  container_name: mysql-8
  environment:
   MYSQL_ROOT_PASSWORD: "123456"
   MYSQL_DATABASE: user_db
  ports:
   - "3306:3306"
  volumes:
   - ~/mysql_data:/var/lib/mysql
```

然后先在我的家目录创建相关文件夹，写入yml文件

```
ellery@ellery-VMware-Virtual-Platform:~/mysql-compose$ vim compose.yml
ellery@ellery-VMware-Virtual-Platform:~/mysql-compose$ cat compose.yml
services:
  mysql:
   image: mysql:8.0
   container_name: mysql-8
   environment:
    MYSQL_ROOT_PASSWORD: "123456"
    MYSQL_DATABASE: user_db
   ports:
    - "3306:3306"
   volumes:
    - /home/ellery/mysql_data:/var/lib/mysql
```

接着运行容器

```
ellery@ellery-VMware-Virtual-Platform:~/mysql-compose$ docker compose up -d
[+] up 2/2
 ✔ Network mysql-compose_default Created
 ✔ Container mysql-8             Started
ellery@ellery-VMware-Virtual-Platform:~/mysql-compose$ docker ps
CONTAINER ID   IMAGE       COMMAND                  CREATED         STATUS         PORTS                                                  NAMES
b6a4968c8647   mysql:8.0   "docker-entrypoint.s…"   4 seconds ago   Up 4 seconds   0.0.0.0:3306->3306/tcp, [::]:3306->3306/tcp, 33060/tcp   mysql-8
ellery@ellery-VMware-Virtual-Platform:~/mysql-compose$
```

连通验证与落库

接下来就是进mysql和建表填表了

1. **先** `docker exec -it mysql-8 /bin/bash` **进bash，再在bash启动mysql：**

```
bash-5.1# mysql -u root -p user_db
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 9
Server version: 8.0.46 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.
```

2. **建表填表**

```
mysql> CREATE TABLE users(
    ->     id int PRIMARY KEY COMMENT"",
    ->     username VARCHAR(50) COMMENT"",
    ->     password varchar(50) COMMENT""
    -> );
Query OK, 0 rows affected (0.01 sec)

mysql> show tables
    -> ;
+------------------+
| Tables_in_user_db |
+------------------+
| users            |
+------------------+
1 row in set (0.00 sec)

mysql> INSERT INTO users (id,username,password) VALUES (1,ellery,123456);
ERROR 1054 (42S22): Unknown column 'ellery' in 'field list'
mysql> INSERT INTO users (id,username,password) VALUES (1,'ellery','123456');
Query OK, 1 row affected (0.00 sec)

mysql> SELECT id,username,password from users;
+----+----------+----------+
| id | username | password |
+----+----------+----------+
|  1 | ellery   | 123456   |
+----+----------+----------+
1 row in set (0.00 sec)
```

删除验证

1. **先退出来看看linux目录里有没有我创建的文件**

```
ellery@ellery-VMware-Virtual-Platform:~$ cd mysql_data
ellery@ellery-VMware-Virtual-Platform:~/mysql_data$ ls
 auto.cnf         binlog.index    client-cert.pem     '#ib_16384_1.dblwr'   ibtmp1          mysql          performance_schema   server-cert.pem   undo_001
 binlog.000001    ca-key.pem      client-key.pem       ib_buffer_pool       '#innodb_redo'   mysql.ibd      private_key.pem      server-key.pem    undo_002
 binlog.000002    ca.pem         '#ib_16384_0.dblwr'   ibdata1              '#innodb_temp'   mysql.sock     public_key.pem       sys               user_db
ellery@ellery-VMware-Virtual-Platform:~/mysql_data$
```

现在可以发现是有的

2. **然后强制删除容器**

```
ellery@ellery-VMware-Virtual-Platform:~/mysql_data$ docker rm -f mysql-8
mysql-8
ellery@ellery-VMware-Virtual-Platform:~/mysql_data$ docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
ellery@ellery-VMware-Virtual-Platform:~/mysql_data$ docker ps -a
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS                     PORTS                                       NAMES
5a2bb293bf7c   nginx          "/docker-entrypoint.…"   23 hours ago   Created                                                        relaxed_easley
e22c3ce40675   nginx          "/docker-entrypoint.…"   23 hours ago   Created                                                        musing_sammet
3fe3e3ed207c   nginx          "/docker-entrypoint.…"   23 hours ago   Exited (0) 23 hours ago                                        practical_carson
1ce56bc4e24d   ubuntu         "/bin/bash"              2 days ago     Exited (0) 2 days ago                                          friendly_wescoff
627474f227b1   nginx:alpine   "/docker-entrypoint.…"   2 days ago     Exited (255) 28 hours ago  0.0.0.0:8080->80/tcp, [::]:8080->80/tcp   silly_yonath
668753cda399   hello-world    "/hello"                 2 days ago     Exited (0) 2 days ago                                          condescending_dijkstra
e5007ec38667   hello-world    "/hello"                 2 days ago     Exited (0) 2 days ago                                          vibrant_sammet
```

3. **再次新建一个mysql容器**



总结

遇到的问题

1. **docker最卡人的绝对是网络连接问题**（，我开始hello-world都跑不出来

   国内没法直连dockerhub，所以会一直连不上

   网上教程普遍是用国内镜像源，可是就我搜到的镜像源就没有一个能用

   最后突然想起代理了，这才能search和pull

   不过不知道为什么系统代理不行得开tun模式

2. **services.volumes must be a mapping**

   这是我写完yml文件后启动容器的报错，意思是volumes得是横杠这种键值对

   但原因是volume我开始没有缩进到mysql: 下面

3. **我误打误撞了一点东西**（没放到题解里

   题目要求挂载到家目录的一个文件夹，我yml那里照做了

   但是我后面忘记了，我想创建一个数据卷在mysql-compose/里，主要是为了不让home乱七八糟

   只能说还好打了yml草稿吧（

如果我是出题人，如何拷打docker学习者？？

个人认为重点在于使用上而非特别底层的概念上，不过基础概念还是要的

1. **镜像和容器这两个最基础概念一定要清楚**
2. **镜像、容器的相关操作**

   拉取、查看、删除////创建、启动、停止、重启、删除
3. **容器端口相关**

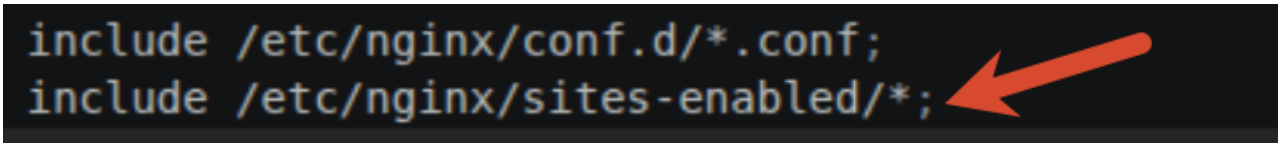
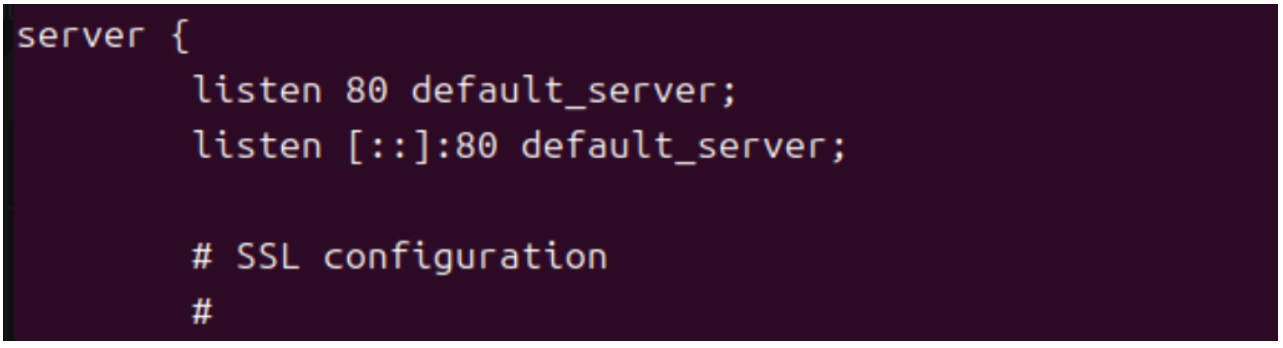
容器如何连接到宿主机和外界?

4. **数据卷挂载**
5. **dockerfile构建镜像**
6. **dockercompose的使用**

~~然而我5和6依然不熟（~

---

part 3

> ## 前言
>
> 这里写一写我遇到的问题
>
> **我第一次启动nginx服务就出错了**
>
> ```
> Job for nginx.service failed because the control process exited with error code.
> See "systemctl status nginx.service" and "journalctl -xeu nginx.service" for details.
> ```
>
> 把错误日志给ai，原因如下：
>
> 绑定80端口失败，因为**80端口已经被占用了**
>
> 大概率是我原来的配置文件还有一个server块写了listen:80，可是我找了没有，原来罪魁祸首是这人：
>
> 
>
>
> 我进去看了看，发现了这个server块
>
> 
>
>
> 简单粗暴解决一下吧，我把那一行include给注释了
>
> **还有第二集**
>
> 我就不应该把网页文件放在家目录，现在权限不够，403了ww
>
> 外部没有读的权限，为了测试我先暂时执行，后面再改回来吧
>
> `chmod 775 /home/ellery`

这里我把index.html放在~/nginx-practice下

btw，在linux用vscode编辑这些真的比vim舒服一万倍

```
ellery@ellery-VMware-Virtual-Platform:~$ code /etc/nginx/nginx.conf
ellery@ellery-VMware-Virtual-Platform:~$ mkdir nginx-practice
ellery@ellery-VMware-Virtual-Platform:~$ cd nginx-practice
ellery@ellery-VMware-Virtual-Platform:~/nginx-practice$ touch index.html
ellery@ellery-VMware-Virtual-Platform:~/nginx-practice$ code index.html
```

接下来编写nginx配置文件

这里我监听8000端口吧

```
server {
listen 8000;

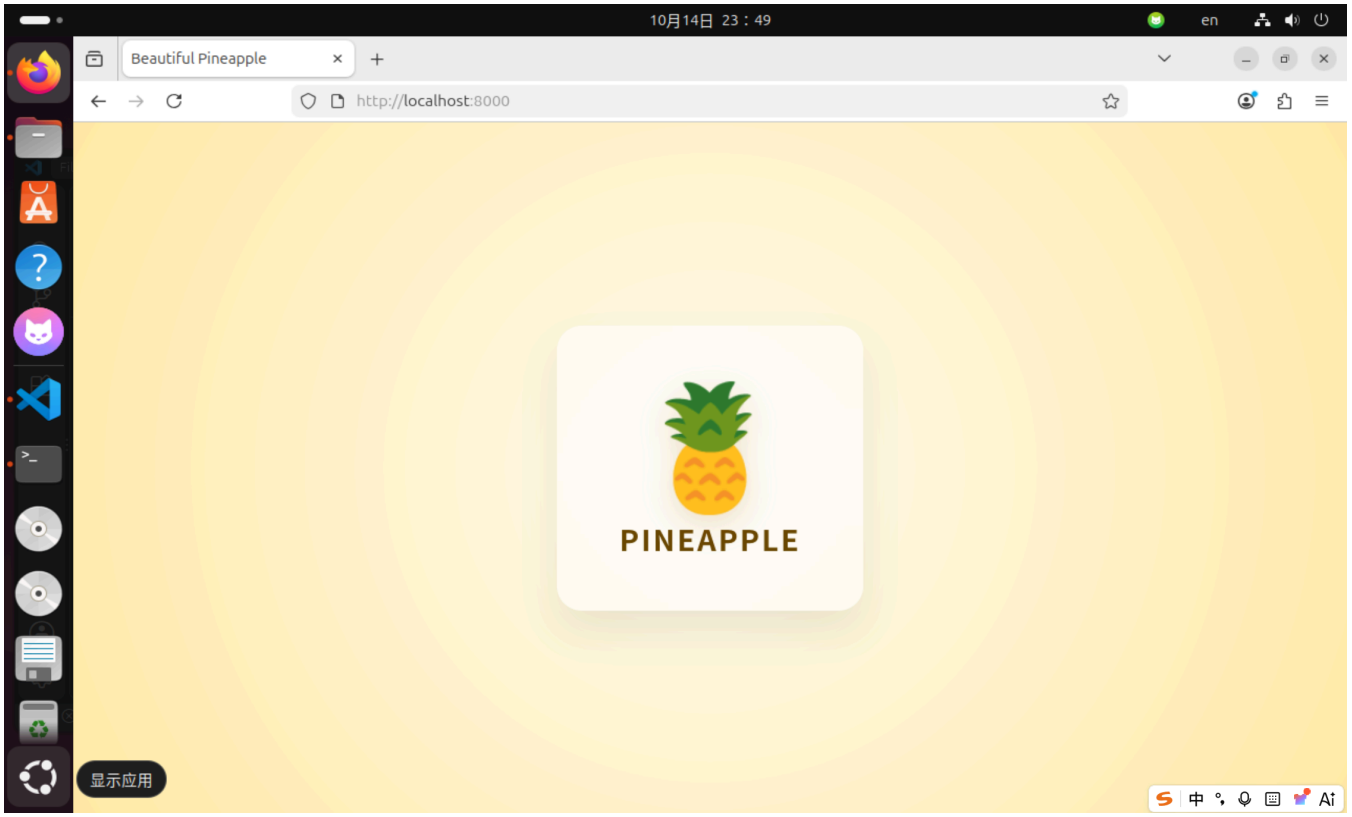
location / {
root /home/ellery/nginx-practice;
index index.html;
}
}
```

启动，直接`sudo systemctl start nginx`

看看现在的8000端口（我开始被网上的神秘小命令骗了），**可以看到，nginx监听了8000**

```
ellery@ellery-VMware-Virtual-Platform:~$ sudo systemctl start nginx
ellery@ellery-VMware-Virtual-Platform:~$ sudo netstat -tulpn | grep 8000
sudo: netstat: 找不到命令
ellery@ellery-VMware-Virtual-Platform:~$ sudo ss -tulpn | grep 8000
tcp   LISTEN 0      511          0.0.0.0:8000       0.0.0.0:*    users:(("nginx",pid=133330,fd=5),("nginx",pid=133329,fd
=5),("nginx",pid=133328,fd=5),("nginx",pid=133327,fd=5),("nginx",pid=133325,fd=5))
```

去浏览器看看



PART 4

相关概念

HTTP是什么？解决了什么问题？

HTTP是超文本传输协议，规定了客户端和服务器之前如何进行请求和响应

解决了**客户端与服务端之间如何按照统一的格式、规则进行请求、响应这样的数据交换**

常见状态码

2开头：成功

3开头：重定向

4开头：客户端问题

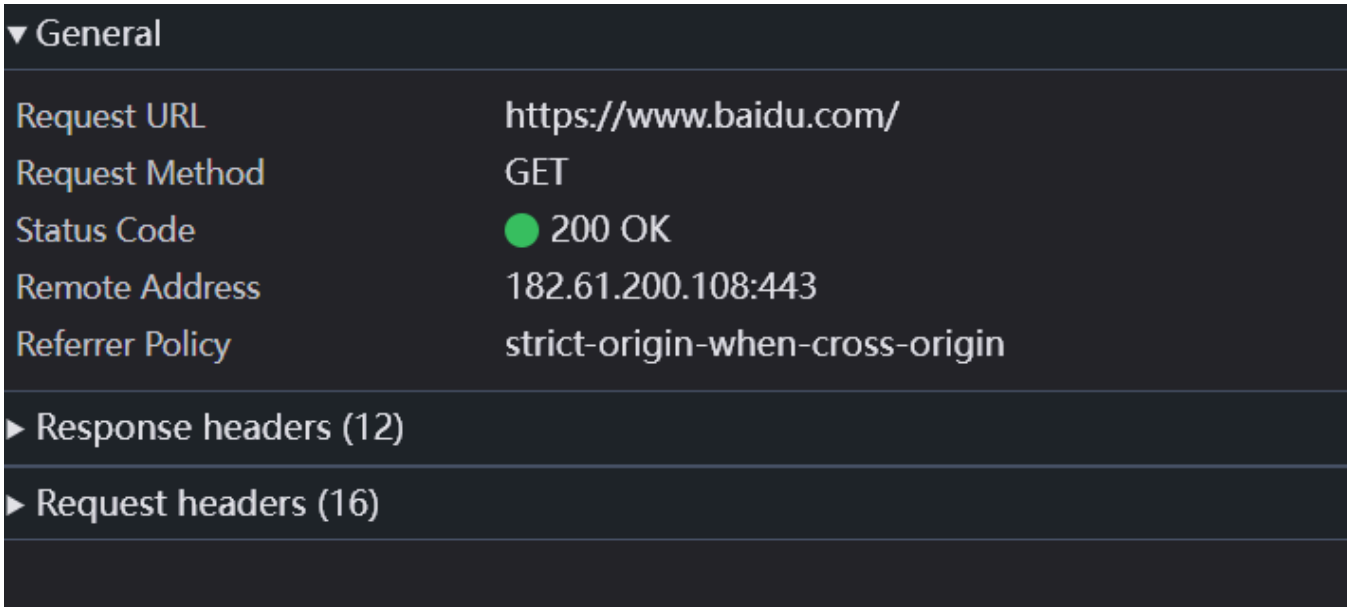
5开头：服务端问题

状态码	含义
200	请求成功

状态码	含义
201	资源创建成功
204	成功，但没有响应体
301	永久重定向
302	临时重定向
400	请求错误
401	未通过身份验证
403	禁止访问
404	资源不存在
500	服务器内部错误
502	反向代理或网关服务异常
503	服务暂不可用

TASK 1

这里看看首页html文档的header



- **请求使用的方法**

  可以看到第二行写了，**GET**

- **请求header里面有什么信息?**

```
GET / HTTP/1.1
Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,image/jxl,image/avif,image/webp,
image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
```

```
Accept-Encoding: gzip, deflate, br, zstd
Accept-Language: zh-CN,zh;q=0.9,en;q=0.8,en-US;q=0.7
Connection: keep-alive
Cookie: *省略
Host: www.baidu.com
Sec-Fetch-Dest: document
Sec-Fetch-Mode: navigate
Sec-Fetch-Site: none
Sec-Fetch-User: ?1
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/155.0.0.0 Safari/537.36
sec-ch-ua: "Google Chrome";v="155", "Chromium";v="155", "Not(A:Brand";v="24"
sec-ch-ua-mobile: ?0
sec-ch-ua-platform: "Windows"
```

1. **请求行（严格说这个应该不属于header）**

```
GET / HTTP/1.1
```

这里指出了**请求方法**、**请求路径**（这里的路径是`/`，表示根路径）和**http协议版本**

2. **请求头主体部分**

请求头里写着请求头字段，用来写说明信息

其中，主要就是这些字段

1. **Host**

`Host: www.baidu.com`，要访问的域名

2. **User-agent**

浏览器、操作系统等**客户端信息**

3. **Accept**

客户端能接受的数据类型

4. **Accept-Language**

偏好语言

5. **Cookie**

用来维持登录、记住用户设置和信息等等

- ## 服务器返回了什么?

服务器的返回体现在响应头里(以下有省略)

```
HTTP/1.1 200 OK
Connection: keep-alive
Content-Encoding: gzip
Content-Security-Policy: frame-ancestors 'self' https://chat.baidu.com http://mirror-chat.baidu.com https://fj-chat.baidu.com https://hba-chat.baidu.com https://hbe-chat.baidu.com https://njjs-chat.baidu.com https://nj-chat.baidu.com https://hna-chat.baidu.com https://hnb-chat.baidu.com http://debug.baidu-int.com https://sai.baidu.com https://mcpstore.baidu.com https://mcpserver.baidu.com https://www.mcpworld.com https://platform-openai.now.baidu.com;
Content-Type: text/html; charset=utf-8
Date: Thu, 15 Oct 2026 16:22:25 GMT
Server: BWS/1.1
Set-Cookie: *
Tr_id: main_0x901d46ae0035c89a
Traceid: 1792081345235426023410384534028928010394
X-Ua-Compatible: IE=Edge,chrome=1
X-Xss-Protection: 1;mode=block
Transfer-Encoding: chunked
```

1. **状态行（这个也不在header内）**

   这里分别是http版本 状态码

2. **响应头的字段（也只写主要的）**

   1. Connection: keep-alive

      不关闭通道让后续请求可以复用

   2. **Content-Type**

      `Content-Type: text/html; charset=utf-8`

      标识出响应体数据类型是html，编码utf-8

   3. **Set-Cookie**

      服务器下发 Cookie 指令，**告诉浏览器保存这条 Cookie**

   4. Content-Encoding

      返回的内容的压缩格式

      > 此外，还有一些比较常用的字段
      >
      > `Content-Length`：响应体字节长度
      >
      > `Location`：重定向跳转地址

当然，服务器还返回了响应体，这里的响应体是html代码，里面写了很多各种图片等等文件的地址

- ## 一次http请求通常包含什么？分别有什么作用？

一次 HTTP 请求通常包含 请求行、请求头、请求体三个部分，其中请求头和请求体用空行隔开

- **请求行**

  请求行可以**指定请求的方法、要请求的路径、http版本**

  `GET / HTTP/1.1`

- **请求头**

请求头里面写的是相应的请求字段，它可以规定请求体的长度、数据类型，指定目标主机等等**附加信息**

- **请求体**

  请求体可以携带需要提交的数据，比如账号等等，它并不是必须的

- **常见 HTTP 请求方法和适用场景**

  这里直接做一个表格

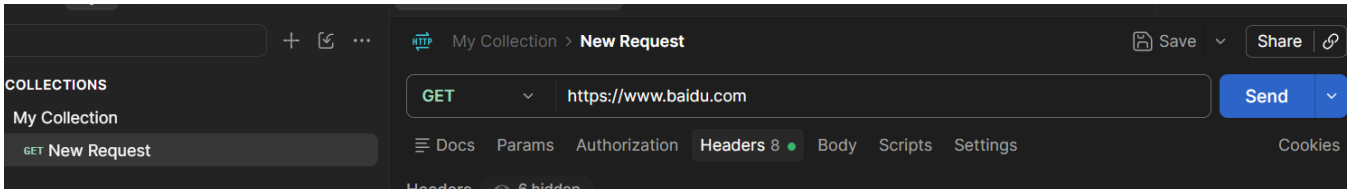
方法	作用	适用场景
GET	获取数据	适合只进行数据获取而不更改文件的情况
POST	发送请求、提交数据、创建资源	登录、提交订单等等需要执行的操作
PUT	创建/整体替换目标资源	更新用户全部资料等等
PATCH	更新部分的资源	修改部分数据，比如头像
DELETE	删除资源	需要删除资源的情况

> 这些是很常用的请求方法，此外还有head等等请求方法，它的作用是获取响应头

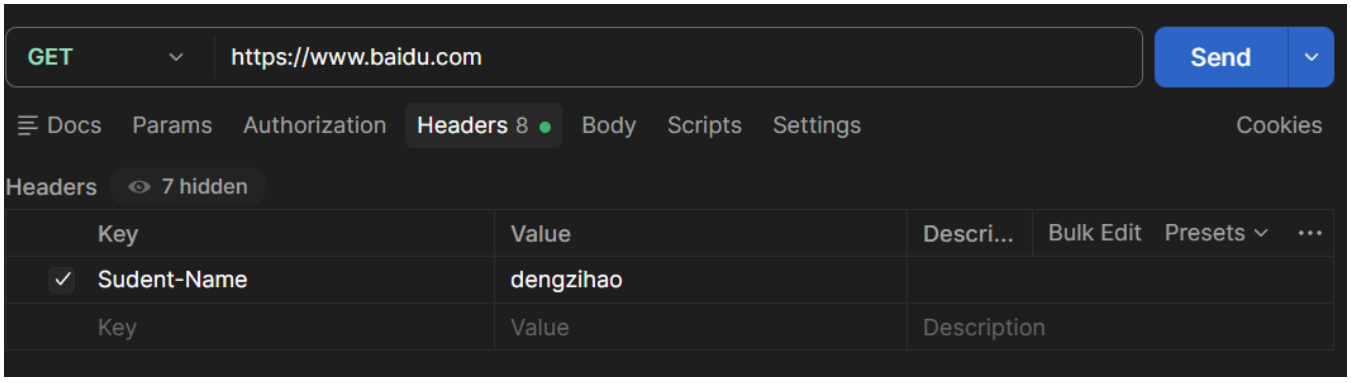
TASK 2

发送一次请求

新建http请求，填写url

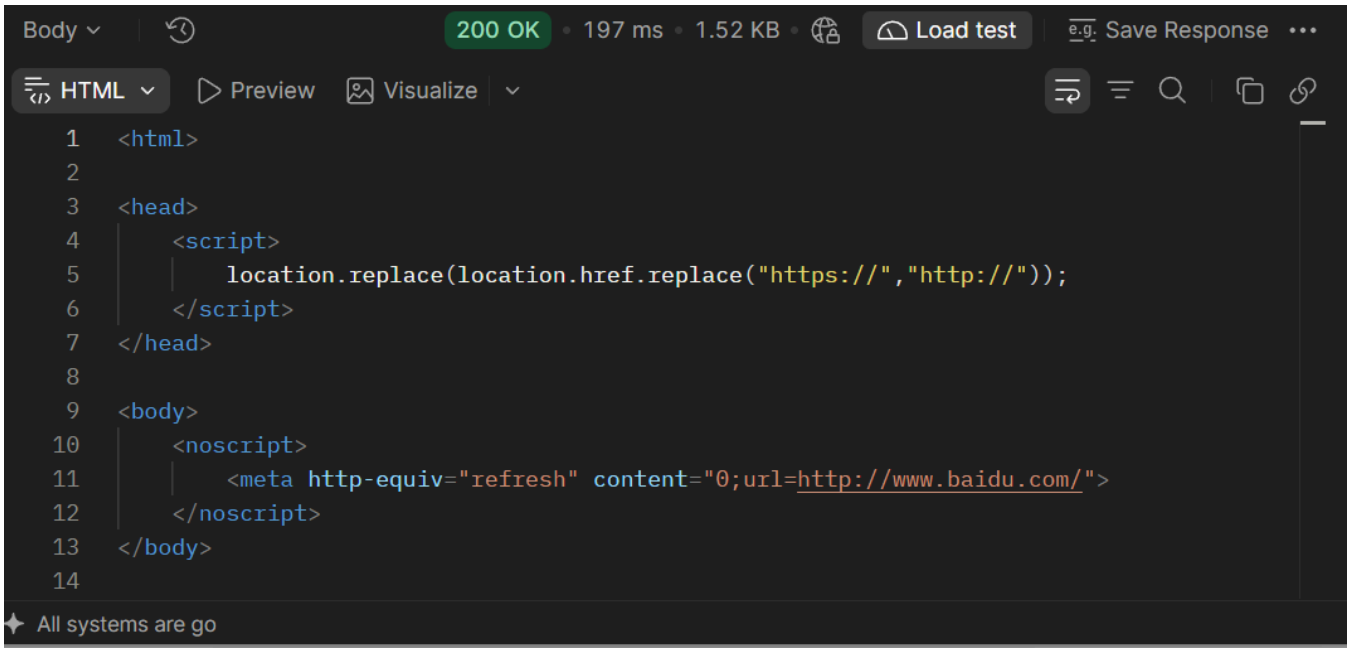


填写请求头的字段参数



send查看返回的结果

可以看到成功了，200 OK



为什么HTTP请求需要header?

http的请求头作用是携带请求的附加信息，让服务器能够明白客户端的请求的各种要求和信息

比如规定能够接受的数据格式，请求体的格式还有各种各样的信息

如果 HTTP 请求没有 Header，服务器还能正常处理所有请求吗？有没有例外情况?

一般来说不行吧，不过的确有特例

一般来说，**很多请求都需要请求头来给服务器提供一些信息**

比如说用JSON来传数据，要在header写**content-type**；http1.1版本还要写**host**字段标明访问的主机

不过如果是简单的请求，比如说像task2这个简单的GET，可以不需要header里的信息

除了Student-Name外，还有哪些常见header？有什么用?

> 上面虽然有一点header总结，但是还是围绕着百度来说的，所以这里再次总结一下

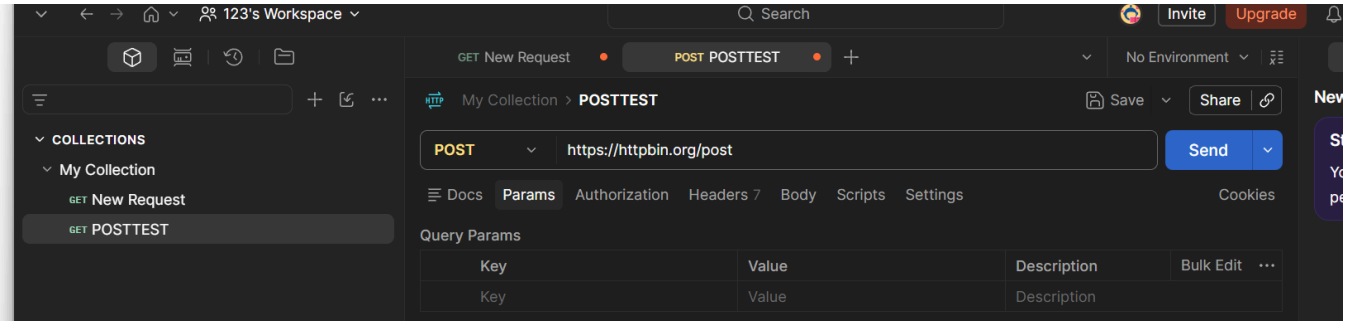
Header	作用
Host	指明要请求的主机
User-Agent	说明当前发送请求的客户端信息，比如浏览器
Accept	告诉服务器客户端能接受哪些文件格式
Accept-Language	设定客户端偏好语言
Content-Type	如果写在请求头，那就规定请求体的正文格式 如果是响应头，那自然就是响应体

Header	作用
Cookie	曲奇饼向服务器发送与当前网站相关的 Cookie 信息，用来保存状态

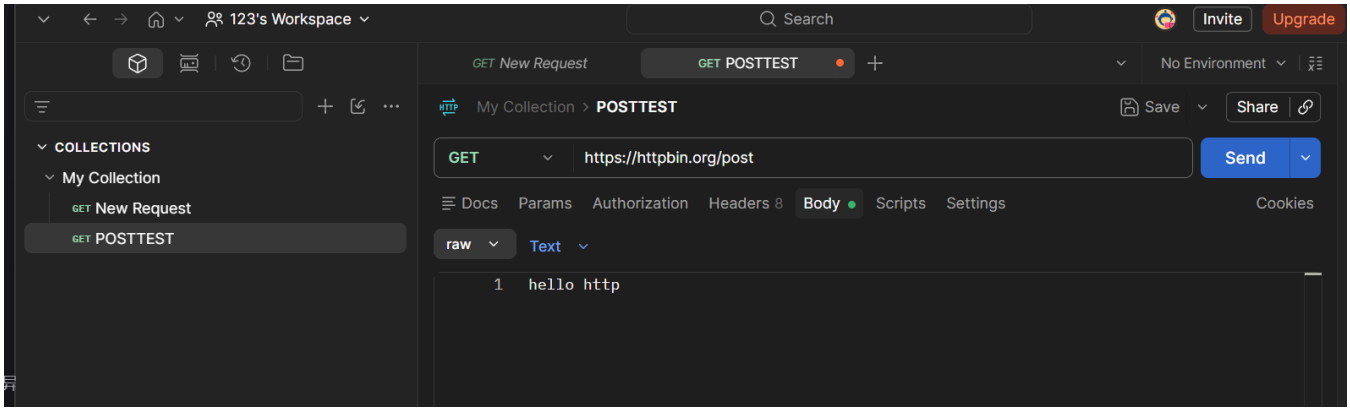
TASK 3

TASK 3-1 发送普通文本

创建新的请求，填写url，选择post

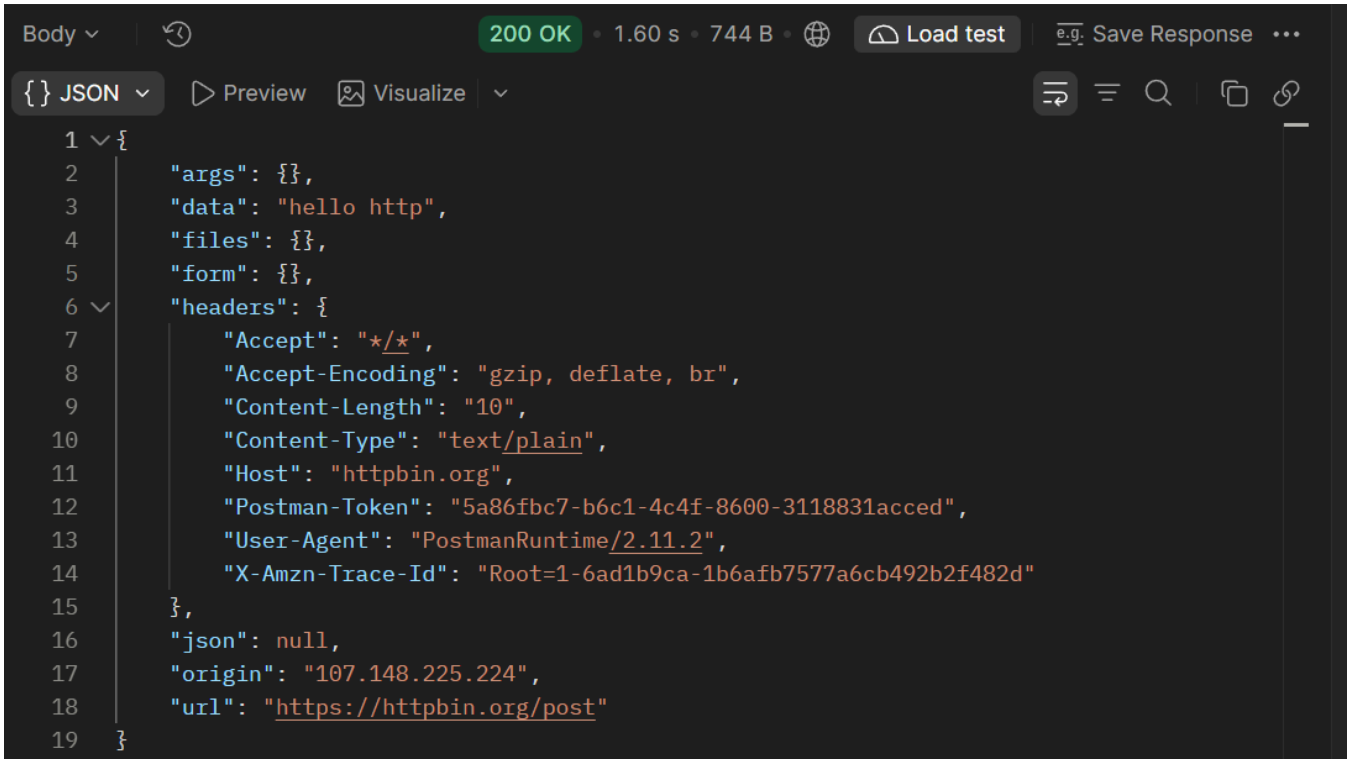


写好body



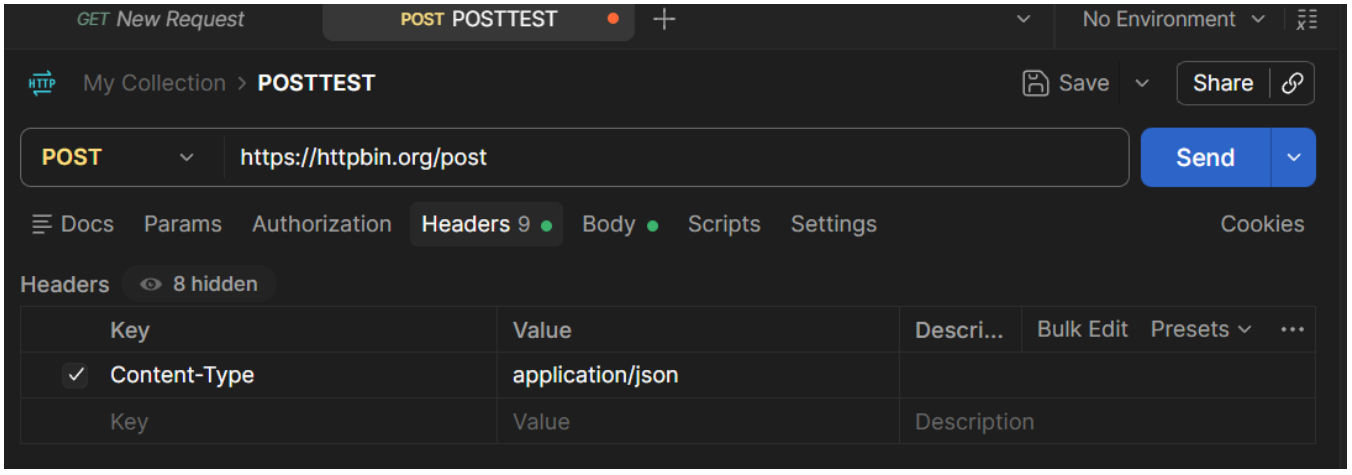
发送请求查看结果

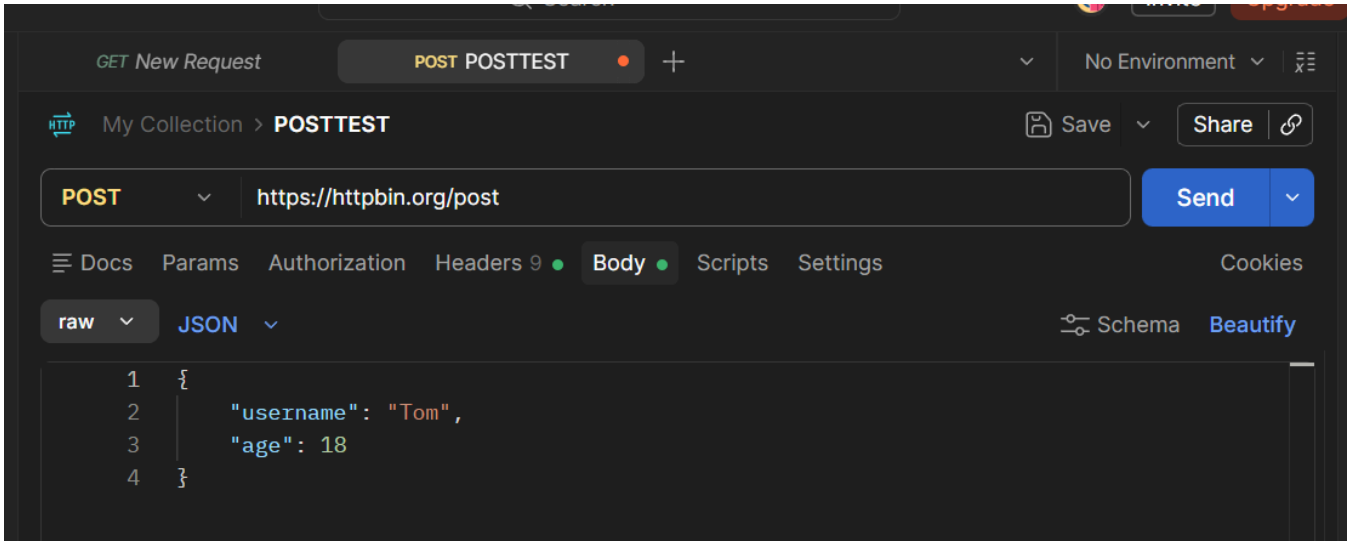
可以看到是一些JSON数据



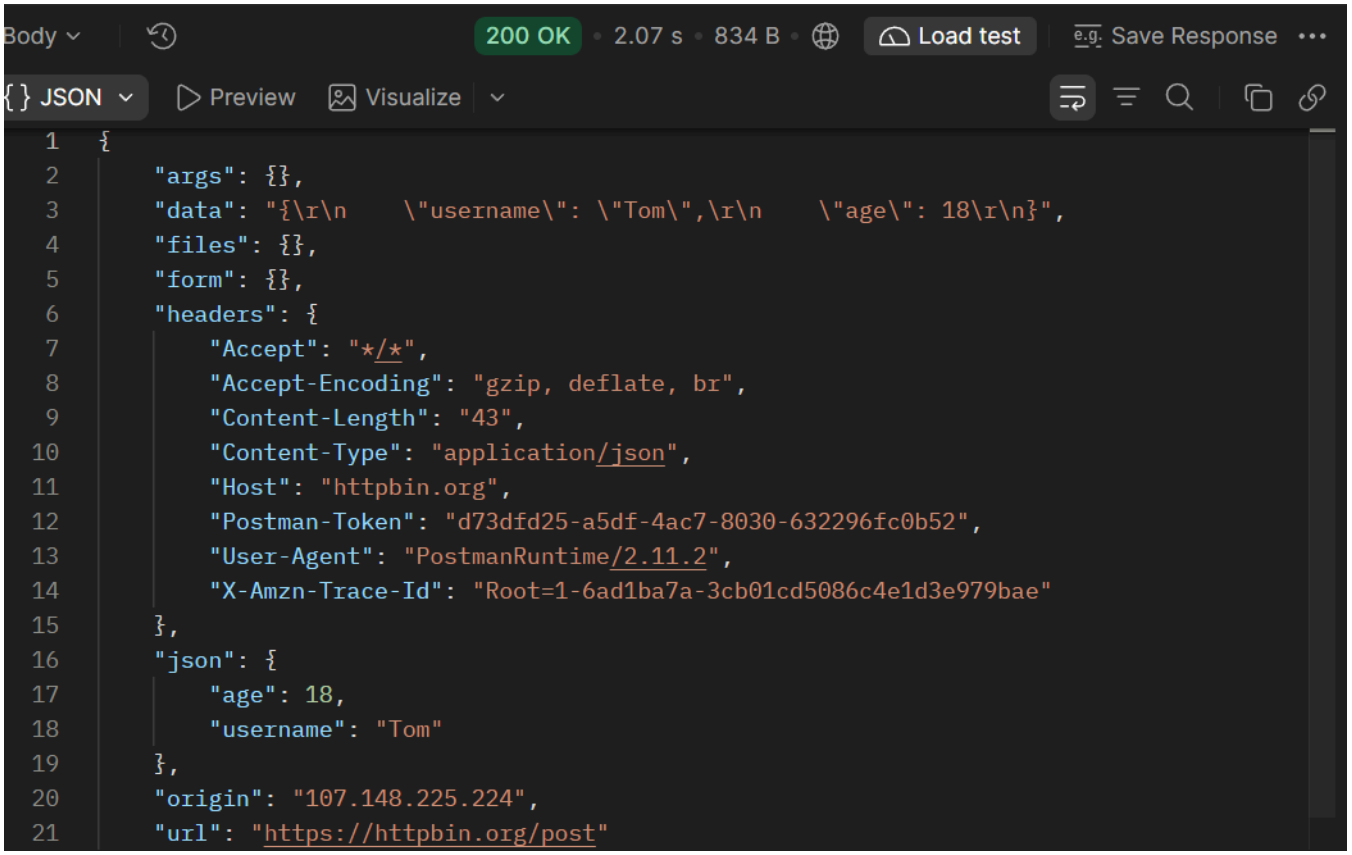
TASK3-2 发送JSON数据

改一下body和header





send查看结果



为什么提交数据时通常使用 POST 请求，而不是 GET 请求?

首先，get的主要作用是获取信息，提交数据主要用post

但是get和post其实都可以提交数据

如果get需要提交数据，**就需要把需要提交的数据写在URL里面**

而post不同，就看刚刚的例子，就能看出**post的数据写在body里**而非放在url，比如

`https://www.baidu.com/s?ie=UTF-8&wd=电子科技大学校草`

如果把请求的数据放在url里，**日志、浏览记录等等地方都可以看到这些数据**

此时倘若这个数据需要一定保密性，那get就非常不好

为什么发送 JSON 数据时需要设置Content-Type?

如果发送请求体里的JSON格式的数据给服务器，服务器是不知道如何解析的，只会把他当成普通文件

所以，需要再请求头里写上contenttype，**让服务器知道正文是JSON格式，正确解析正文**（从上图的data字段可以看到解析成功了）

如果不设置 Content-Type，服务器还能收到数据吗？还能正确解析数据吗?

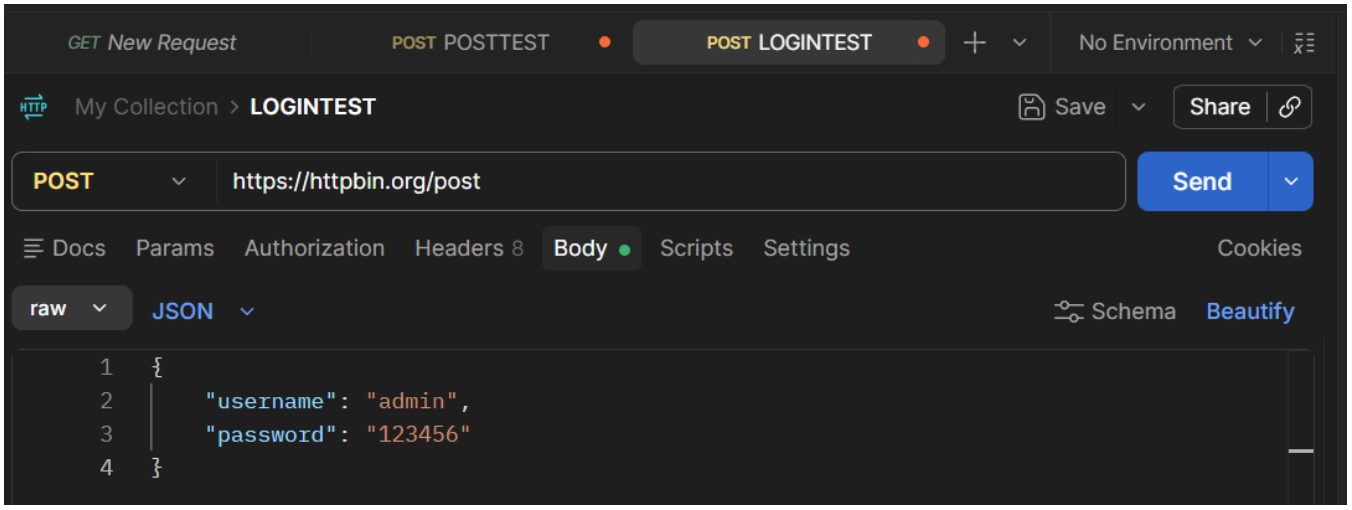
能收到数据但是不能正确解析

正如上文所说，post请求发送时自然会把数据发给服务器

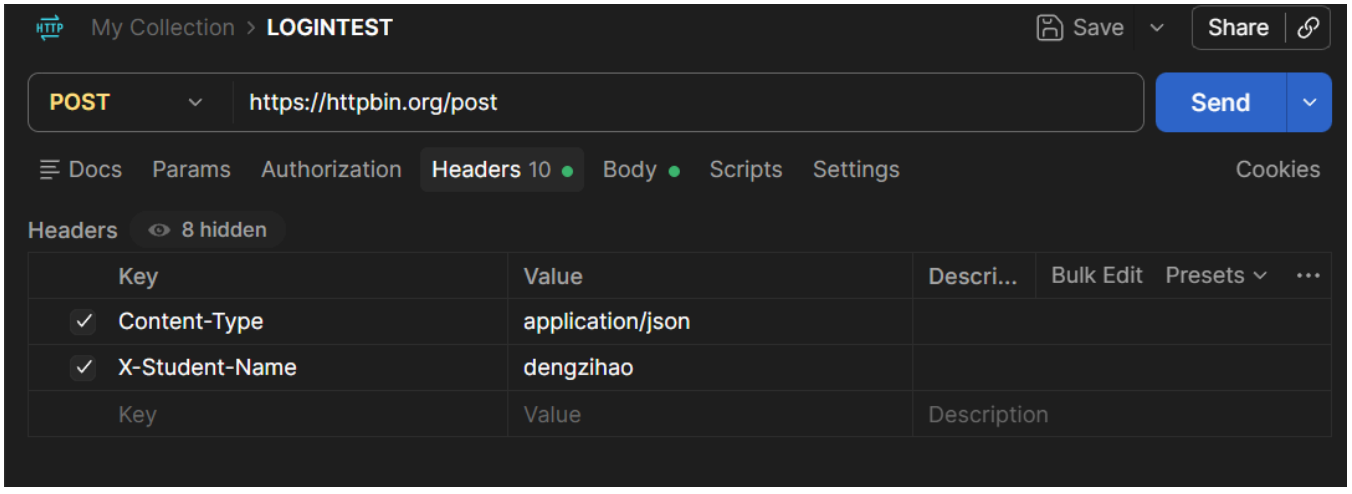
但是如果没设置content-type，倘若是json这样的特殊文本格式，服务器不会按照json来特殊对待，这样解析数据就失败了

TASK 4

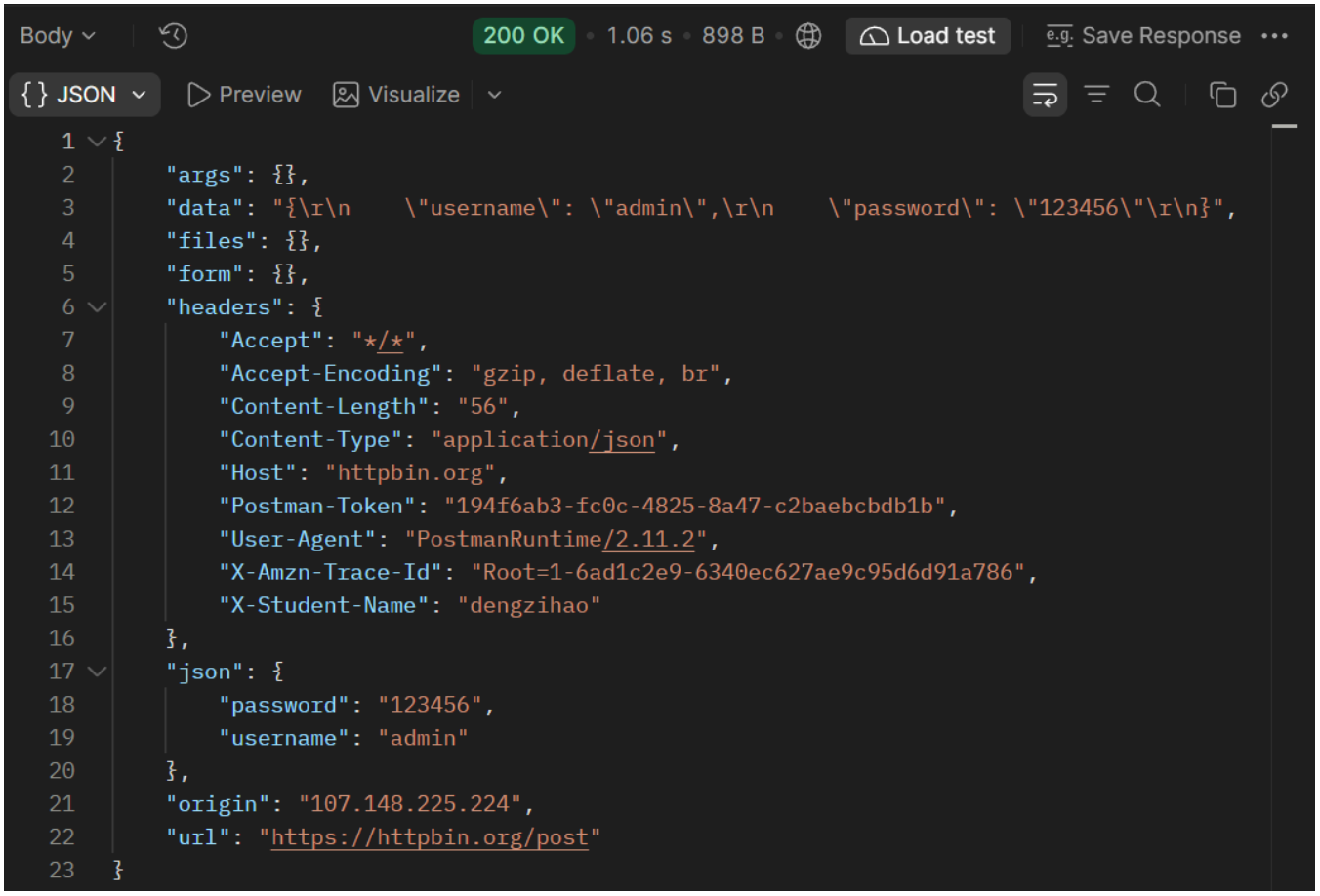
设置方法、URL、body



设置header



查看结果



为什么登录请求通常使用 POST，而不是 GET?

和上面同理，登录请求一般输入的都是账号、密码这些及其私密的内容，如果使用get就要写在url中，**暴露在日志、记录下，让数据不安全**；如果写在post，数据在请求体中，不会有这种烦恼

为什么需要设置 Content-Type: application/json?

这个问题依旧和上面重复

我在请求体里填写的用户名和密码是JSON格式的，所以需要写一个Content-Type请求头，让**服务器知道以JSON解析我正文里的数据**

用户名和密码是否应该直接放在 URL 中？为什么?

肯定不行，因为放在url中就是**在链接后面问号?加上数据**，会让数据停**留在日志和浏览记录，很不安全**

拓展思考题

1. **5次验证码**
    - **为什么“获取验证码”通常使用的是 POST 请求而不是 GET 请求?**

      和登录同理呀，获取验证码一般来说提交的数据是手机号，这个比用户名更私密

      还是那句话，写在url会让数据有日志记录，不安全

- **如果用户多次重复发送同一个 HTTP 请求，为什么 GET 请求通常不会造成问题，而 POST 请求可能会产生问题?**

  这里有一个概念“**幂等性**”，和数据安全和重复请求的影响有关，意思是同一个请求执行一次或多次，对服务器产生的最终效果都一样

  btw，这里的安全并不是上面说的暴露数据，**而是你做出这个方法是否会对数据做出改动影响数据**

  GET方法是幂等、安全的，它作用就是获取数据，**一般不会让服务器执行修改资源相关的操作**

  POST不一样，它会发送请求、提交数据，让服务器执行相应操作，由于**post不幂等**，**如果5次重复发送post请求，那就会让服务器5次处理这个请求，做出5次相关操作**

- **后端可以采用什么手段防止这种因为 HTTP 请求重复发送带来的问题?**

  后端处理这种问题的方法主要是**幂等键和限流**

  幂等键和mysql数据库的那个主键很类似，它也是**唯一的标识**，让客户端的请求携带一个唯一的标识，让后端记录这个标识有没有处理过，如果有，会拒绝这一次请求

  限流则**类似于前端的禁用60s按钮操作**，不过更深层一点，他会**限制客户端一定时间内的请求次数**

> 此外个人认为还可以让多次请求不影响最终结果，我记得之前多次收验证码可以重复的，不过我也不知道咋实现（

2. **为什么要设计确认重新提交表单弹窗???**

   这个弹窗也是防止上面那个问题的一个措施

   比如我高考报志愿，在那个网站填好了学校，但是网太卡了提交一直不显示成功界面，此时我刷新就会弹出来这个窗口

   这是因为我开始已经发起了一个post请求（不一定成功了），浏览器也不知道我这个post成功没

   这时候我刷新，带来的结果会是再提交一个post请求

   如果上一个没成功，那还行；如果成功了，就会出现第一个问题的结果：多次提交post请求

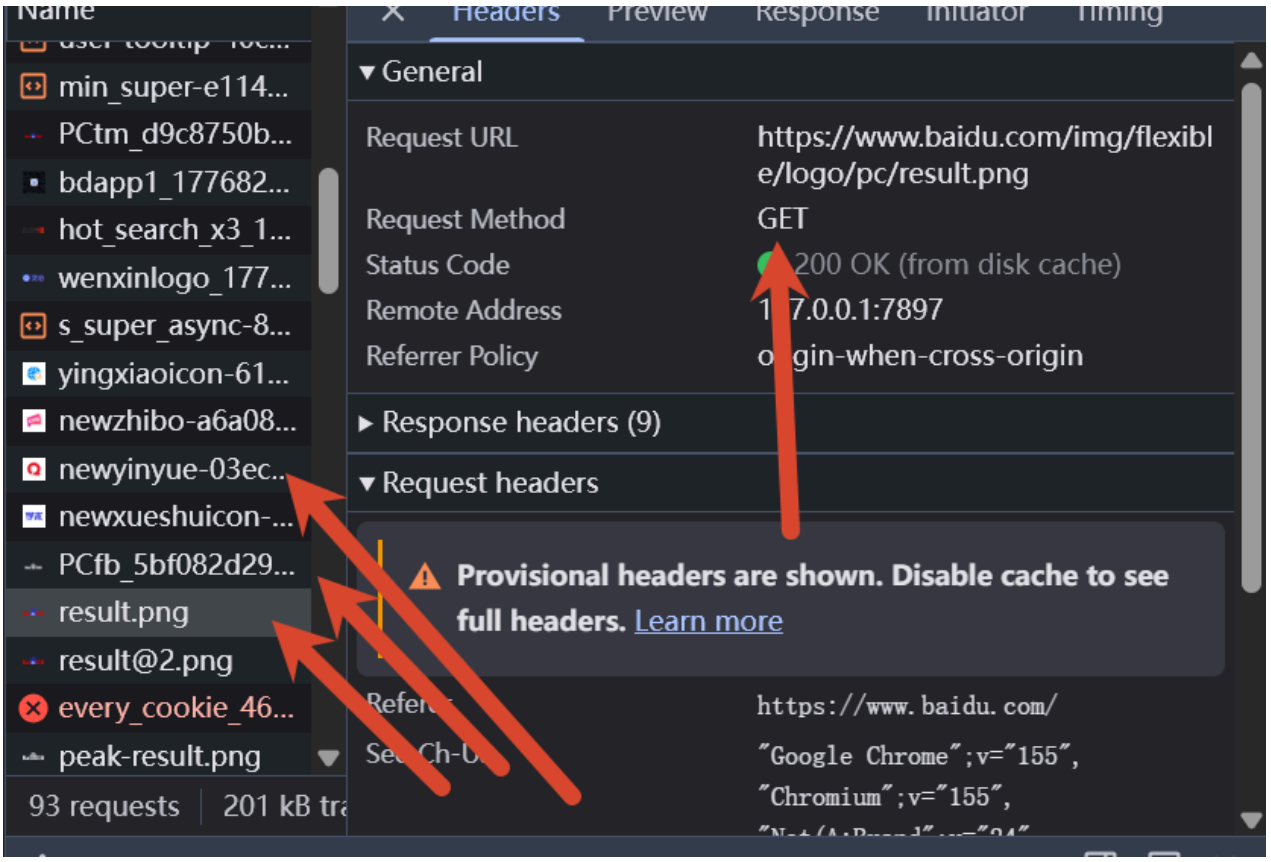
   所以，这个弹窗可以让用户确认是否再次提交，防止有多个post请求被提交产生错误，增加一点容错

3. **一个网页上的图片、CSS、JavaScript 文件，是不是也通过 HTTP 请求获取的?**

   我觉得是

   就拿f12看百度来说，不止是最上面一条，还有很多小图片、js文件组成了这个大ui，这些文件点进去可以看到都是有对应的请求报文的

   要获取网页上的图片、CSS、JavaScript 文件等等资源，**就需要发送多个请求（这些请求可以是同时的）**


Name
min_super-e114...
PCtm_d9c8750b...
bdapp1_177682...
hot_search_x3_1...
wenxinlogo_177...
s_super_async-8...
yingxiaoicon-61...
newzhibo-a6a08...
newyinyue-03ec...
newxueshuicon-...
PCfb_5bf082d29...
result.png
result@2.png
every_cookie_46...
peak-result.png
93 requests
201 kB tra
Headers
Preview
Response
Initiator
Timing
General
Request URL
https://www.baidu.com/img/flexible/logo/pc/result.png
Request Method
GET
Status Code
200 OK (from disk cache)
Remote Address
7.0.0.1:7897
Referrer Policy
gin-when-cross-origin
Response headers (9)
Request headers
Provisional headers are shown. Disable cache to see full headers. Learn more
https://www.baidu.com/
"Google Chrome";v="155",
"Chromium";v="155",